Historical Manuscript Fragment Reconstruction & Evidence Explorer

Project Type: Software Development Project

Project Description and Organizational Use: I propose to develop an application that helps libraries, archivists, museums, and history researchers organize incomplete or separated manuscript pages by comparing textual, visual, and descriptive clues. The system will accept manuscript page images, text, or metadata and generate possible page sequences rather than claiming to recover missing content. It will explain the evidence behind each proposed arrangement so researchers can review and compare possible reconstructions.

How I Arrived at This Proposal: I initially used Copilot to generate three possible 14-week software development projects based on my original interest in helping college students manage assignments, deadlines, and study schedules. Copilot suggested the Smart Assignment and Deadline Manager, AI-Assisted Study Planner, and Collaborative Academic Success Hub. After reviewing these ideas and discussing them with a classmate, I realized that I was more interested in working with historical documents and solving a problem related to organizing incomplete or separated manuscript pages. Because this interest was different from the original Copilot topic, I used Gemini to explore whether a manuscript-focused software project could be realistic for a 14-week semester project. Gemini helped me identify the idea of comparing textual, visual, and metadata-based evidence to suggest possible manuscript page sequences. I then refined the idea myself based on the course requirements, available public manuscript data, the Python tools I could use, and what could realistically be completed by one student during the semester. Therefore, the final proposal was selected and refined by me based on my interests, the course requirements, available data, and project feasibility.

Organizational Requirements: The system will compare page numbers, keywords, transcribed text, visual/layout features, image similarity, and available manuscript metadata. Candidate pages will be represented as a graph, and ordering constraints will eliminate impossible sequences. Python will be used with Pandas for metadata processing, NumPy for numerical calculations, OpenCV and scikit-image for image processing and visual similarity, RapidFuzz for approximate text matching, NetworkX for candidate relationship graphs, OR-Tools for constraint-based page ordering, SQLite for local storage, and Streamlit for the interactive application. Each library will support a specific part of the fragment matching, ordering, storage, or visualization process.

Data, Expected Advantages, and Resources: The primary data will come from publicly accessible digitized manuscript collections from the British Library and Digital Bodleian, using available manuscript page images and catalogue metadata; the Internet Archive will be an additional source where suitable materials are openly accessible. For evaluation, I will create controlled test cases by taking manuscripts with known page sequences and artificially separating pages, removing page numbers, or hiding selected metadata. This will allow the system's proposed sequences to be compared with the known original order. The project will reduce the manual effort involved in comparing possible page arrangements while allowing researchers to inspect the evidence supporting each result. The application will run locally on a normal laptop and will not require proprietary databases or specialized hardware.

My Prompt 1:

I want to develop a unique Python software development project involving historical documents or manuscripts. I want the system to solve a real problem rather than simply display or search documents. Can you suggest a project involving algorithms, text analysis, image analysis, and publicly available manuscript data?

Gemini Response:

One possible project is a Historical Manuscript Fragment Reconstruction system. The application could help researchers determine the possible ordering of separated manuscript pages by comparing textual clues, page metadata, visual layout, writing characteristics, and relationships between fragments. Rather than claiming to reconstruct missing content, the system could rank possible page sequences and explain the evidence supporting each arrangement.

My Prompt 2:

How could I technically implement a Historical Manuscript Fragment Reconstruction system in Python? I want to combine text similarity, image comparison, metadata, graph algorithms, and constraints, and I need publicly available data for testing.

Gemini Response:

The system could represent each manuscript fragment as a record containing its image, text, page identifiers, metadata, and extracted features. Text similarity could identify fragments with related content, image-processing techniques could compare visual layout and page characteristics, and metadata could provide additional clues. A graph could represent possible relationships between fragments, while a constraint solver could eliminate impossible page orders. The remaining candidate sequences could be ranked using a combined evidence score. Publicly digitized manuscript collections from libraries and archives could provide the source

data, while artificially separating known manuscript sequences would provide controlled test cases.

I used **Gemini** to explore and technically refine the historical manuscript project idea, including its feasibility, algorithms, and potential public data sources.